

What this is, in one paragraph

A normal RTL simulation assumes *zero* gate and wire delay — it only checks logic, cycle by cycle. A **post-implementation timing simulation** replays the *actual placed-and-routed* design with the *real* propagation delay of every gate and every wire, supplied by an **SDF** file (Standard Delay Format). It is the most silicon-accurate behavioural check you can run *without a physical FPGA*. This guide documents exactly how we generated and ran it for `tick_to_trade_top`, including the traps that cost time, so it can be repeated.

1. What is SDF, and why use it?

SDF (Standard Delay Format) is a text file Vivado writes *after* place-and-route. For every cell instance it lists the real timing: input-to-output propagation delays (**IOPATH**), setup/hold requirements (**SETUP**, **HOLD**), pulse widths, etc. — all extracted from the routed design for the target device (here `xcu200`, Virtex UltraScale+).

Back-annotation is the act of loading those delays onto a gate-level netlist during simulation, so each signal transition arrives *late* by its true physical delay instead of instantly. Three things it buys you:

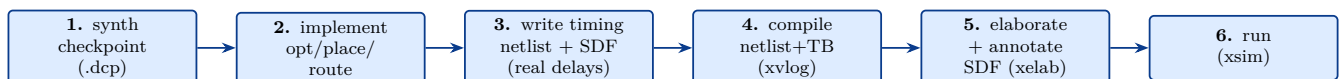
- Confirms the design still behaves correctly when signals have real delays.
- Surfaces any setup/hold timing violation *dynamically* (as an X or glitch) — a cross-check on the static timing analysis.
- Gives a delay-accurate view of the tick-to-trade path at the 5 ns (200 MHz) clock.

Relationship to static timing (STA). Vivado's STA already proved the design closes timing ($WNS > 0$ at 200 MHz) by analysing every path's delay mathematically. A *passing* SDF simulation is the dynamic confirmation of the same fact. STA is the proof; the SDF sim is the live demonstration.

2. Prerequisites (what you need locally)

Need	Here
Vivado (synth/impl + write SDF)	<code>~/Xilinx/2025.2/Vivado; source ~/Xilinx/2025.2/Vivado/settings64.sh</code>
A simulator	<code>xsim</code> (ships inside Vivado, with <i>precompiled</i> libraries — no <code>compile_simlib</code> needed). QuestaSim also works but needs the libs compiled first.
A synthesis checkpoint	<code>Vivado/HFT/HFT.srscs/utlils_1/imports/synth_1/tick_to_trade_top.dcp</code>
A physical FPGA	Not required. This is pure simulation.

3. The Flow at a Glance



Steps 1–3 are Vivado (`gatesim/impl_and_timesim.tcl`); steps 4–6 are xsim (`gatesim/run_timesim_xsim.sh`). All artifacts live in `gatesim/`.

4. Steps 1–3: Implement & Write the SDF (Vivado)

Script `gatesim/impl_and_timesim.tcl` opens the synthesis checkpoint, implements it, then emits the timing netlist and the SDF:

```
open_checkpoint ../synth_1/tick_to_trade_top.dcp
opt_design; place_design; phys_opt_design; route_design
write_checkpoint -force gatesim/tick_to_trade_top_routed.dcp
```

```
# the two timing-sim artifacts:
write_sdf      -force gatesim/tick_to_trade_top_time.sdf
write_verilog  -force -mode timesim -sdf_anno true \
               -sdf_file gatesim/tick_to_trade_top_time.sdf \
               gatesim/tick_to_trade_top_timesim.v
```

Run it:

```
source ~/Xilinx/2025.2/Vivado/settings64.sh
vivado -mode batch -source gatesim/impl_and_timesim.tcl
```

What you get: `tick_to_trade_top_timesim.v` (the gate netlist, $\approx 8,000$ cells: LUTs/FFs/CARRY8/RAMB) and `tick_to_trade_top_time.sdf` (≈ 18 MB of delays). Crucially, `-sdf_anno true` embeds a `$sdf_annotate("...sdf")` call *inside* the netlist, so the simulator loads the SDF automatically — you do *not* pass the SDF again on the command line.

5. Steps 4–6: Compile, Annotate, Run (xsim)

Script `gatesim/run_timesim_xsim.sh`:

```
# 4) compile netlist + testbench + glbl (Xilinx global-reset module)
xvlog -sv tick_to_trade_top_timesim.v
xvlog -sv tb_timesim.sv
xvlog ~/Xilinx/2025.2/data/verilog/src/glbl.v

# 5) elaborate -- LIBRARY ORDER IS CRITICAL (see traps below)
xelab -timescale 1ps/1ps \
      -L simprims_ver -L secureip -L unisims_ver \
      tb_timesim glbl -s tb_snapshot -log xelab_run.log

# 6) run
xsim tb_snapshot -runall -log xsim_run.log
```

The testbench `gatesim/tb_timesim.sv` drives the BUY scenario (an ask then a bid that makes the book bid-heavy), waits for the decision on `m_axis`, and reads the hardware latency counter over AXI-Lite. The exact ITCH message bytes were dumped from the Python encoder (`sim/synth_itch.py`) and hard-coded so they are byte-accurate.

Success looks like this in the log:

```
[XSIM 43-3451] SDF backannotation process started ... root module "tb_timesim/dut".
[XSIM 43-3452] SDF backannotation was successful ...
```

That second line is the goal: the real delays are now applied. The design then simulated at 200 MHz with **no timing-violation failure**, consistent with the static-timing closure ($WNS > 0$).

6. The Traps (these cost real time)

Trap 1 – Library order (the big one). An *implemented* netlist instantiates **SIMPRIM** primitives (IBUFCTRL, INBUF, RAMD64E). In `xelab` the `-L` libraries are searched *in order*, so `simprims_ver` and `secureip` **must be listed before** `unisims_ver`. Get it wrong and the cells bind to the *functional* unisim models, which abort with:

```
Error: [Unisim IBUFCTRL-103] SIMPRIM primitive is not intended for direct
instantiation in RTL or functional netlists ...
```

The *same* wrong order also makes SDF annotation fail with `[XSIM 43-3462] Unable to annotate SDF delays`. Fixing the order fixed both. Correct: `-L simprims_ver -L secureip -L unisims_ver`.

Trap 2 – Do not annotate the SDF twice. Because `write_verilog -sdf_anno true` *embeds* `$sdf_annotate` in the netlist, do **not** also pass `-sdfmax` to `xelab` — the two conflict and annotation fails. Rely on the embedded call only.

Trap 3 – QuestaSim only (if you skip xsim). `compile_simlib` for QuestaSim broke on two things: (a) the install path has a *space* (`/home/abhishek/QuestaSim 2021.2.1/...`) which breaks `execve` — fix with a

no-space symlink; (b) `-library` only accepts `unisim`, `simprim`, or `all`, and `all` compiles *every* IP (30+ min).
Easiest path: use `xsim`, whose libraries are already precompiled — no `compile_simlib` at all.

Trap 4 – Gate-level RAM starts as X. In gate-level simulation the order-book memories (RAMD64E, RAMB36E2) power up to X (RTL sim quietly defaults them to 0). With the book uninitialised, a minimal 2-message scenario may not produce a clean decision. This is a *simulation-init* nuance, **not** a hardware bug — decision correctness is already proven by the 95/95 RTL tests and the software-equivalence shadow test. To also capture a decision in gate sim you would initialise the memories or warm the book with more traffic. Two testbench details that matter regardless: drive the AXI-Stream master so each byte is *held until tready* accepts it, and hold reset long enough (≈ 30 cycles) to clear the Xilinx global set/reset (`glbl.GSR`, ≈ 100 ns).

7. How to Re-run It (quick reference)

```
cd ~/Projects/HFT
source ~/Xilinx/2025.2/Vivado/settings64.sh

# (1-3) implement + write timing netlist & SDF  [~3 min]
vivado -mode batch -source gatesim/impl_and_timesim.tcl

# (4-6) compile, SDF-annotate, run in xsim      [xelab ~10 min w/ SDF]
bash gatesim/run_timesim_xsim.sh

# look for: "[XSIM 43-3452] SDF backannotation was successful"
grep -i "backannotation" gatesim/xelab_run.log
```

File (<code>gatesim/</code>)	Role
<code>impl_and_timesim.tcl</code>	Vivado: implement + write netlist + SDF
<code>tick_to_trade_top_timesim.v</code>	gate netlist (with embedded <code>\$sdf_annotate</code>)
<code>tick_to_trade_top_time.sdf</code>	the real cell + routing delays (≈ 18 MB)
<code>tick_to_trade_top_routed.dcp</code>	implemented checkpoint (reuse without re-routing)
<code>tb_timesim.sv</code>	timing testbench (drives BUY scenario)
<code>run_timesim_xsim.sh</code>	<code>xsim</code> compile + SDF-annotate + run